28

Going Offline

Users expect applications to operate seamlessly with unreliable network connections. If your mobile application can't cope with transient network issues, your users will use a different app. When there's no network, you have to persist data locally on the device. Alternatively, perhaps your app doesn't even require network access, in which case you'll still need to store data locally.

In this chapter, you'll learn how to do those three things with React Native. First, you'll learn how to detect the state of the network connection. Second, you'll learn how to store data locally. Lastly, you'll learn how to synchronize local data that's been stored due to network problems once it comes back online.

In this chapter, we'll cover the following topics:

- Detecting the state of the network
- Storing application data

- Synchronizing application data

Technical requirements

You can find the code files for this chapter on GitHub at **https://github.com/PacktPublishing/React-and-React-Native-5E/tree/main/Chapter28**.

Detecting the state of the network

If your code tries to make a request over the network while disconnected using `fetch()`, for example an error will occur. You probably have error-handling code in place for these scenarios already, since the server could return some other type of error.

However, in the case of connectivity trouble, you might want to detect this issue before the user attempts to make network requests.

There are two potential reasons for proactively detecting the network state. The first one is to prevent the user from performing any network requests until you've detected that the app is back online. To do that, you can display a friendly message to the user stating that, since the network is disconnected, they can't do anything. The other possible benefit of early net-

work state detection is that you can prepare to perform actions offline and sync the app state when the network is connected again.

Let's look at some code that uses the `NetInfo` utility from the `@react-native-community/netinfo` package to handle changes in network state:

```
const connectedMap = {
  none: "Disconnected",
  unknown: "Disconnected",
  cellular: "Connected",
  wifi: "Connected",
  bluetooth: "Connected",
  ethernet: "Connected",
  wimax: "Connected",
  vpn: "Connected",
  other: "Connected",
} as const;
```

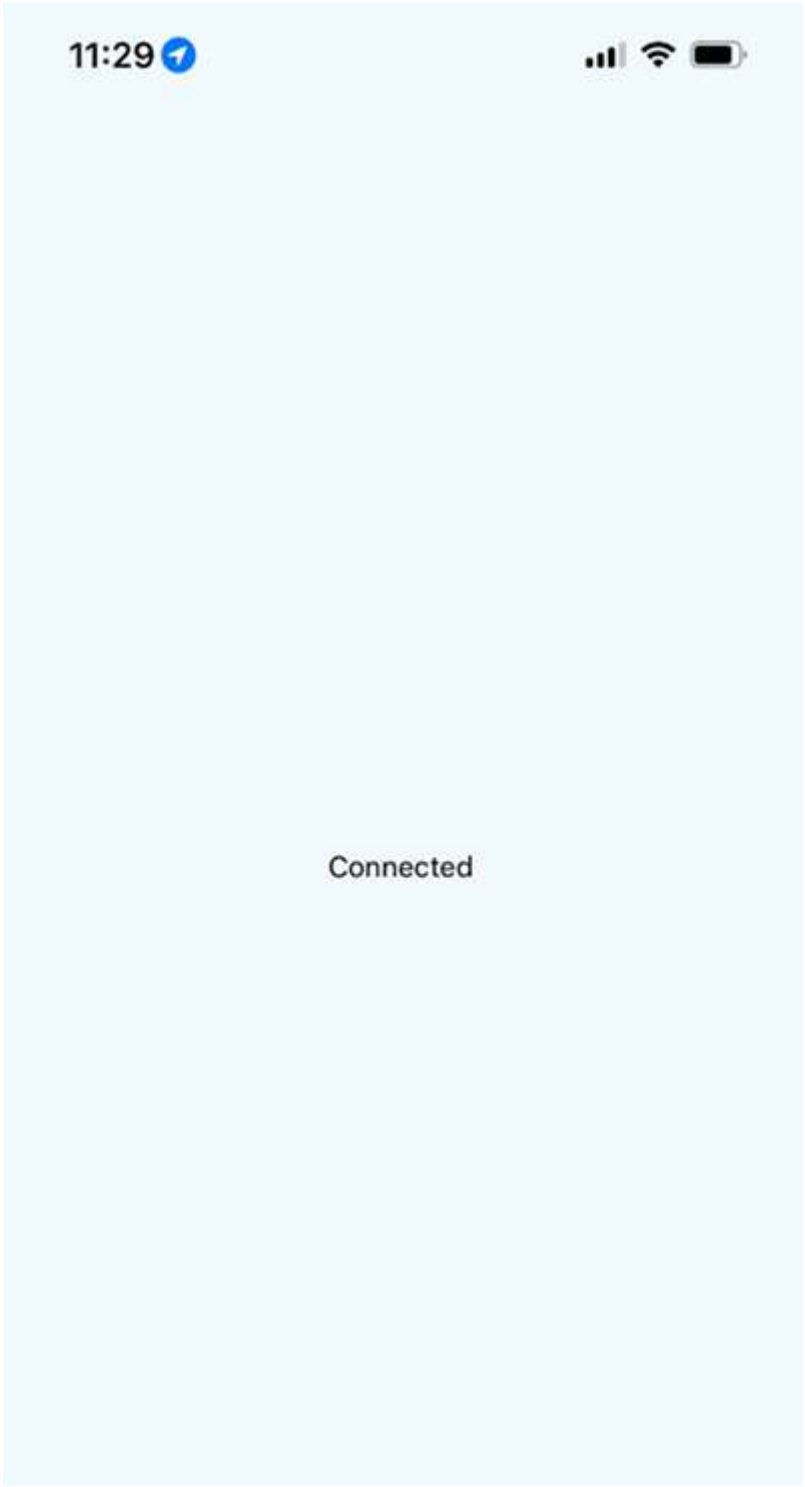
`connectedMap` covers all connection states and will help us to render them on the screen. Let's now see the `App` component:

```
export default function App() {
  const [connected, setConnected] = useState("");
  useEffect(() => {
    function onNetworkChange(connection: NetInfoState) {
      const type = connection.type;
```

```
        setConnected(connectedMap[type]);
      }
      const unsubscribe = NetInfo.addEventListener(onNetworkChange);
      return () => {
        unsubscribe();
      };
    }, []);
    return (
      <View style={styles.container}>
        <Text>{connected}</Text>
      </View>
    );
  }
```

This component will render the state of the network based on the string values in `connectedMap`. The `onNetworkChange` event of the `NetInfo` object will cause the connected state to change.

For example, when you run this app for the first time, the screen might look like this:


11:29
Connected

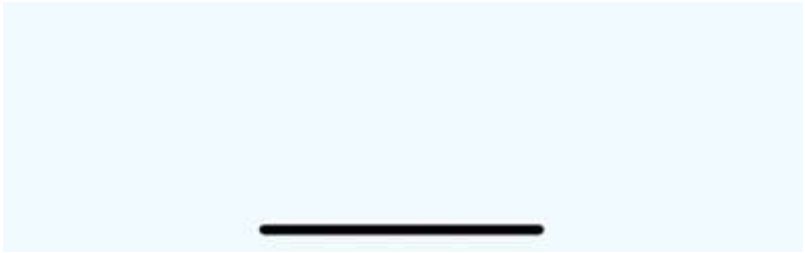
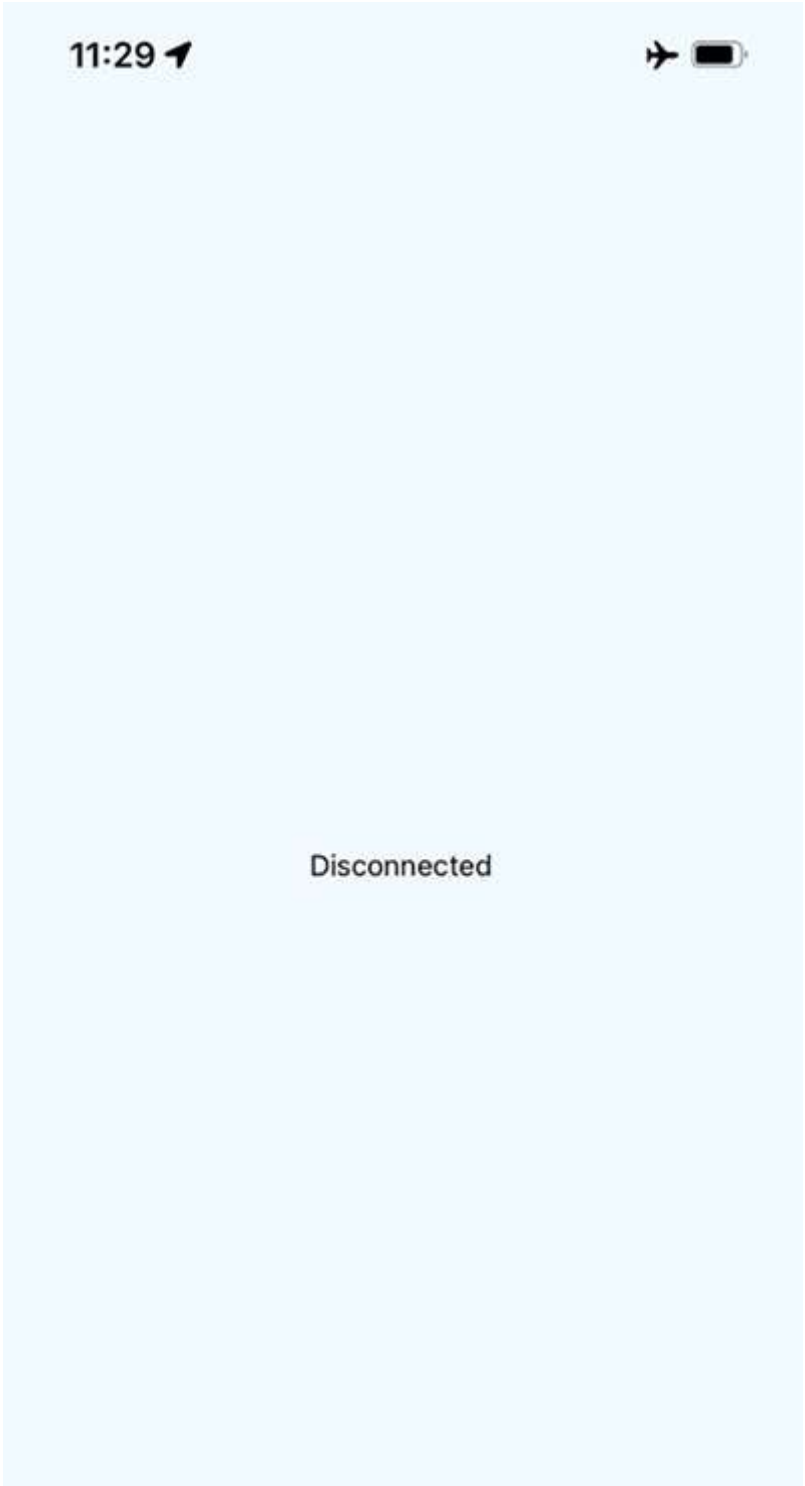


Figure 28.1: Connected state

Then, if you turn off networking on your host machine, the network state will change on the emulated device as well, causing the state of our application to change, as follows:


11:29
Disconnected

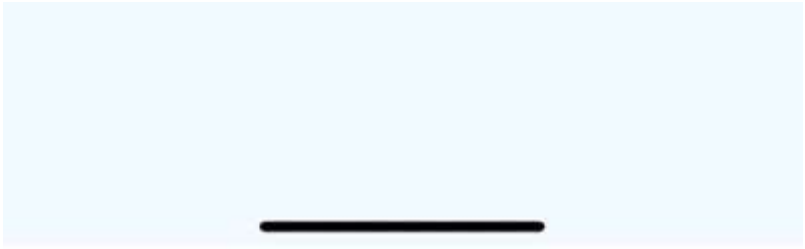


Figure 28.2: Disconnected state

This is how you can use network state detection in the app. As discussed, together with showing the message, you can use network state to prevent users from making API requests. Another valuable approach would be to save user inputs locally until the network gets back online. Let's explore it in the next section.

Storing application data

To store data on the device, there is a special cross-platform solution called `AsyncStorage` API. It works the same on both the iOS and Android platforms. You would use this API for applications that don't require any network connectivity in the first place or to store data that will eventually be synchronized using an API endpoint once a network becomes available.

To install the `async-storage` package, run the following command:

```
npx expo install @react-native-async-storage/async-storage
```

Let's look at some code that allows the user to enter a `key` and a `value` and then stores them:

```
export default function App() {
  const [key, setKey] = useState("");
  const [value, setValue] = useState("");
  const [source, setSource] = useState<KeyValuePair[]>([]);
```

The `key`, `value`, and `source` values will handle our state. To save it in `AsyncStorage`, we need to define functions:

```
  function setItem() {
    return AsyncStorage.setItem(key, value)
      .then(() => {
        setKey("");
        setValue("");
      })
      .then(loadItems);
  }
  function clearItems() {
    return AsyncStorage.clear();
  }
  async function loadItems() {
    const keys = await AsyncStorage.getAllKeys();
    const values = await AsyncStorage.multiGet(keys);
```

```
    setSource([...values]);
  }
  useEffect(() => {
    loadItems();
  }, []);
```

We've defined handlers to save values from the inputs, clear `AsyncStorage`, and load saved items when we start the app. Here's the markup that's rendered by the `App` component:

```
  return (
    <View style={styles.container}>
      <Text>Key:</Text>
      <TextInput
        style={styles.input}
        value={key}
        onChangeText={(v) => {
          setKey(v);
        }}
      />
      <Text>Value:</Text>
      <TextInput
        style={styles.input}
        value={value}
        onChangeText={(v) => {
          setValue(v);
        }}
      />
      <View style={styles.controls}>
```

```
          <Button label="Add" onPress={setItem} />
          <Button label="Clear" onPress={clearItems} />
        </View>
```

The markup in the preceding code block is represented as inputs and buttons to create, save, and delete items. Next, we will render the list of items with the `FlatList` component:

```
        <View style={styles.list}>
          <FlatList
            data={source.map(([key, value]) => ({
              key: key.toString(),
              value,
            }))}
            renderItem={({ item: { value, key } }) => (
              <Text>
                {value} ({key})
              </Text>
            )}
          />
        </View>
      </View>
    );
```

Before we walk through what this code is doing, let's take a look at the following screen, since it'll explain most of what we're going to cover when storing application data:

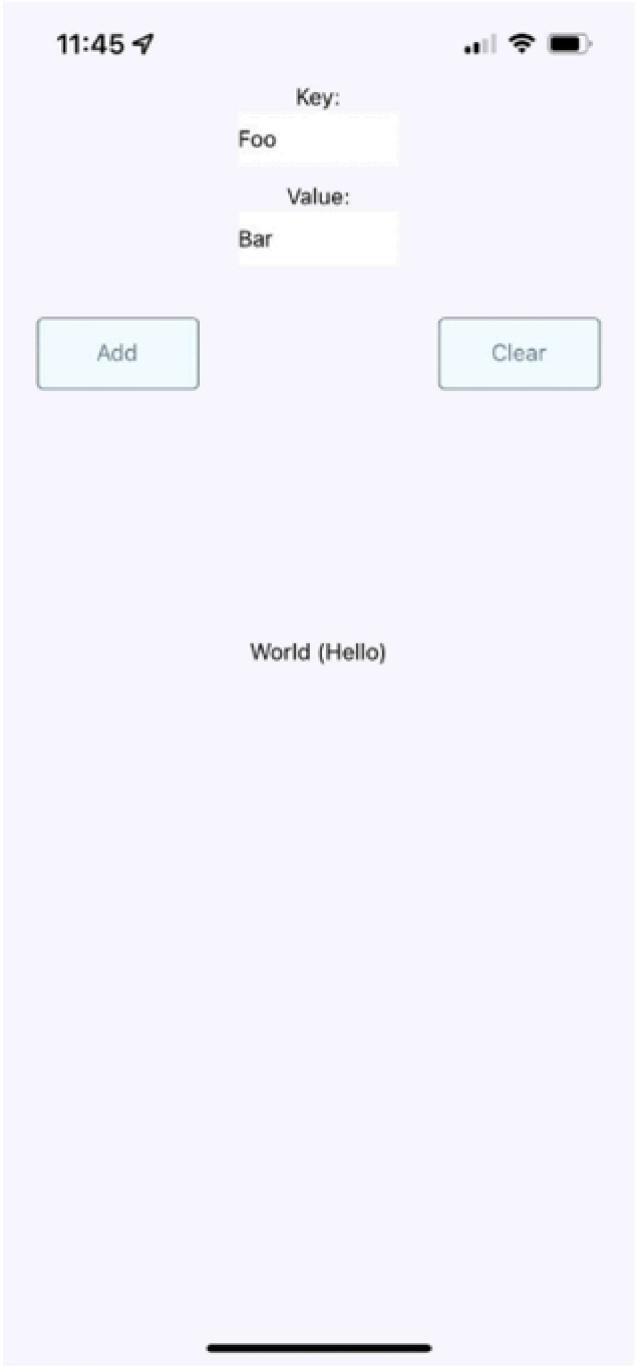

11:45
Key:
Foo
Value:
Bar
Add
Clear
World (Hello)

Figure 28.3: Storing application data

As you can see in *Figure 28.3*, there are two input fields and two buttons. The fields allow the user to enter a new `key` and `value`. The **Add** button allows the user to store this key-value pair locally on their device, while the **Clear** button clears any existing items that have been stored previously.

The `AsyncStorage` API works the same for both iOS and Android. Under the hood, `AsyncStorage` works very differently, depending on which platform it's running on. The reason React Native is able to expose the same storage API on both platforms is due to its simplicity: it's just key-value pairs. Anything more complex than that is left up to the application developer.

The abstractions that you've created around `AsyncStorage` in this example are minimal. The idea is to set and get items. However, even straightforward actions like this deserve an abstraction layer. For example, the `setItem()` method you've implemented here will make the asynchronous call to `AsyncStorage` and update the item's state once that has been completed. Loading items is even more complicated because you need to get the keys and values as two separate asynchronous operations.

The reason we do this is to keep the UI responsive. If there are pending screen repaints that need to happen while data is being written to disk, preventing those from happening by blocking them would lead to a suboptimal user experience.

In the next section, you'll learn how to synchronize data that's been stored locally while the device is offline with remote services once the device comes back online.

Synchronizing application data

So far in this chapter, you've learned how to detect the state of a network connection and how to store data locally in a React Native application. Now, it's time to combine these two concepts and implement an app that can detect network outages and continue to function.

The basic idea is to only make network requests when you know for sure that the device is online. If you know that it isn't, you can store any changes in the state locally. Then, when you're back online, you can synchronize those stored changes with the remote API.

Let's implement a simplified React Native app that does this. The first step is to implement an abstraction that sits between the React components and the network calls that store data. We'll call this module `store.ts`:

```
export function set(key: Key, value: boolean) {
  return new Promise((resolve, reject) => {
    if (connected) {
      fakeNetworkData[key] = value;
      resolve(true);
    } else {
      AsyncStorage.setItem(key, value.toString()).then(
        () => {
          unsynced.push(key);
          resolve(false);
        },
        (err) => reject(err)
      );
    }
  });
}
```

The `set` method depends on the `connected` variable, and depending on whether there is an internet connection or not, it handles the different logic. Actually, the `get` method also follows the same approach:

```
export function get(key?: Key): Promise<boolean | typeof fakeNetworkData> {
  return new Promise((resolve, reject) => {
    if (connected) {
      resolve(key ? fakeNetworkData[key] : fakeNetworkData);
    } else if (key) {
      AsyncStorage.getItem(key)
```

```
          .then((item) => resolve(item === "true"))
          .catch((err) => reject(err));
      } else {
        AsyncStorage.getAllKeys()
          .then((keys) =>
            AsyncStorage.multiGet(keys).then((items) =>
              resolve(Object.fromEntries(items) as any)
            )
          )
          .catch((err) => reject(err));
      }
    });
  }
```

This module exports two functions, `set()` and `get()`. Their jobs are to set and get data, respectively. Since this is just a demonstration of how to sync between local storage and network endpoints, this module just mocks the actual network with the `fakeNetworkData` object.

Let's start by looking at the `set()` function. It's an asynchronous function that will always return a promise that resolves to a Boolean value. If it's true, it means that you're online and that the call over the network was successful. If it's false, it means that you're offline, and `AsyncStorage` was used to save the data.

The same approach is used with the `get()` function. It returns a promise that resolves a Boolean value that indicates the state of the network. If a key argument is provided, then the value for that key is looked up. Otherwise, all the values are returned, either from the network or from `AsyncStorage`.

In addition to these two functions, this module does two other things:

```
NetInfo.fetch().then(
  (connection) => {
    connected = ["wifi", "unknown"].includes(connection.type);
  },
  () => {
    connected = false;
  }
);
NetInfo.addEventListener((connection) => {
  connected = ["wifi", "unknown"].includes(connection.type);
  if (connected && unsynced.length) {
    AsyncStorage.multiGet(unsynced).then((items) => {
      items.forEach(([key, val]) => set(key as Key, val === "true"));
      unsynced.length = 0;
    });
  }
});
```

It uses `NetInfo.fetch()` to set the connected state. Then, it adds a listener to listen for changes in the network state. This is how items that were saved locally when you were offline become synced with the network when it's connected again.

Now, let's check out the main application that uses these functions:

```
export default function App() {
  const [message, setMessage] = useState<string | null>(null);
  const [first, setFirst] = useState(false);
  const [second, setSecond] = useState(false);
  const [third, setThird] = useState(false);
  const setters = new Map([
    ["first", setFirst],
    ["second", setSecond],
    ["third", setThird],
  ]);
```

Here, we have defined the state variables that we will use in the `Switch` components:

```
  function save(key: Key) {
    return (value: boolean) => {
      set(key, value).then(
        (connected) => {
          setters.get(key)?.(value);
          setMessage(connected ? null : "Saved Offline");
```

```
        },
        (err) => {
          setMessage(err);
        }
      );
    };
  }
```

The `save()` function helps us to reuse logic in a different `Switch` component. Next, we have the `useEffect` hook to get saved data when the page renders for the first time:

```
  useEffect(() => {
    NetInfo.fetch().then(() =>
      get().then(
        (items) => {
          for (let [key, value] of Object.entries(items)) {
            setters.get(key)?.(value);
          }
        },
        (err) => {
          setMessage(err);
        }
      )
    );
  }, []);
```

Next, let's take a look at the final markup of the page:

```
    return (
      <View style={styles.container}>
        <Text>{message}</Text>
        <View>
          <Text>First</Text>
          <Switch value={first} onValueChange={save("first")} />
        </View>
        <View>
          <Text>Second</Text>
          <Switch value={second} onValueChange={save("second")} />
        </View>
        <View>
          <Text>Third</Text>
          <Switch value={third} onValueChange={save("third")} />
        </View>
      </View>
    );
```

The job of the `App` component is to save the state of three `Switch` components, which is difficult when you're providing the user with a seamless transition between online and offline modes. Thankfully, your `set()` and `get()` abstractions, which are implemented in another module, hide most of the details from the application's functionality.

Note, however, that you need to check the state of the network in this module before you attempt to load any items. If you

don’t do this, then the `get()` function will assume that you’re offline, even if the connection is fine.

Here’s what the app looks like:

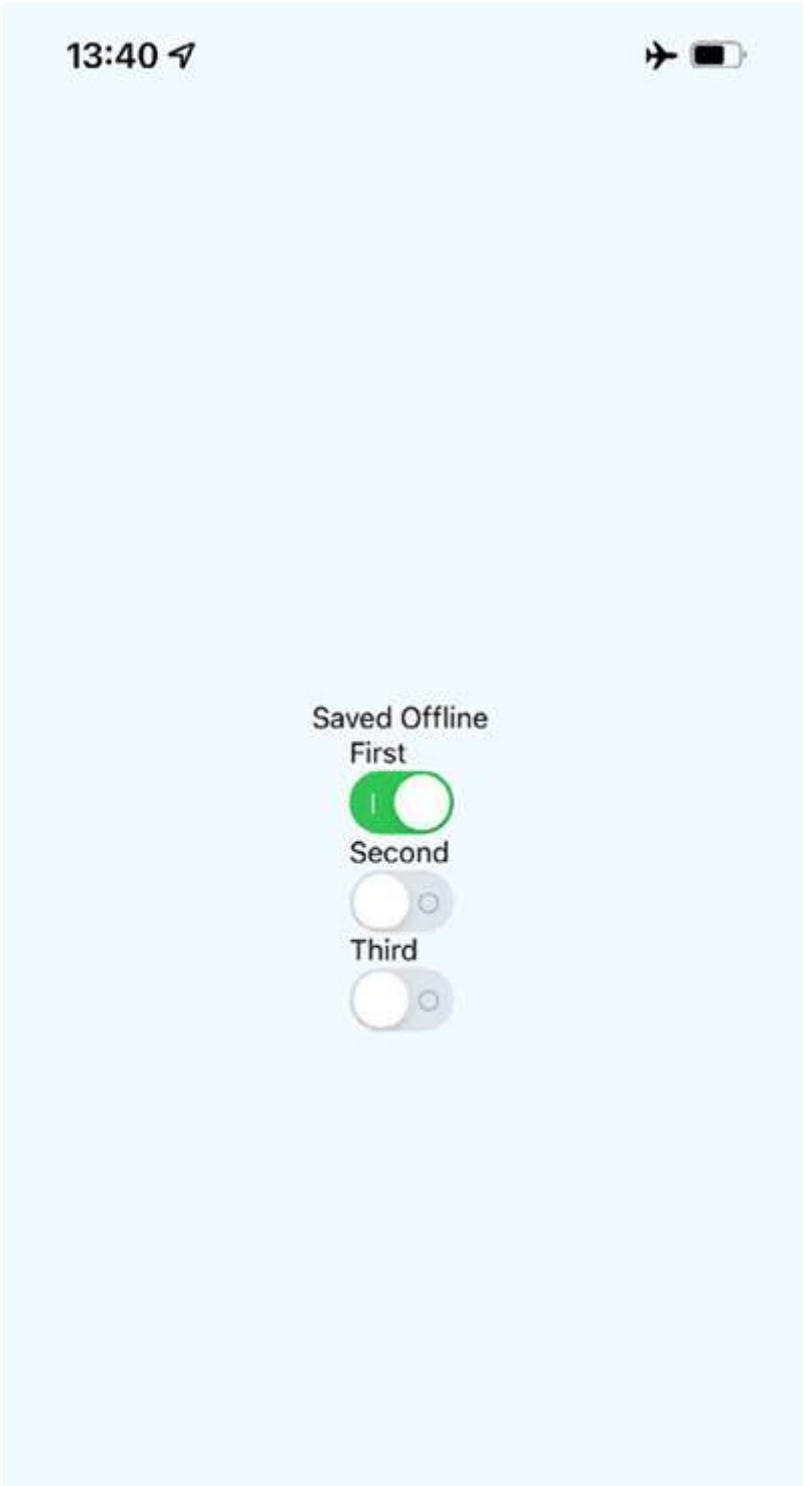

13:40
Saved Offline
First
Second
Third

Figure 28.4: Synchronizing application data

Note that you won't actually see the **Saved Offline** message until you change something in the UI.

Summary

This chapter introduced us to storing data offline in React Native applications. The main reason we would want to store data locally is when the device goes offline and our app can't communicate with a remote API. However, not all applications require API calls, and `AsyncStorage` can be used as a general-purpose storage mechanism. We just need to implement the appropriate abstractions around it.

We also learned how to detect changes in the network state of React Native apps. It's important to know when the device has gone offline so that our storage layer doesn't make pointless attempts at network calls. Instead, we can let the user know that the device is offline and then synchronize the application state when a connection is available.

In the next chapter, we'll learn how to import and use UI components from the NativeBase library.

Join us on Discord!

Read this book alongside other users and the authors themselves. Ask questions, provide solutions to other readers, chat with the authors, and more. Scan the QR code or visit the link to join the community.

https://packt.link/ReactAndReactNative5e